`socket() → bind() → listen() → accept()`

A REST API in C

Lessons learned from building an API for CRUD operations — from raw sockets up, one byte at a time.

`Todo API · GET · POST · GET/:id · PATCH · PUT · DELETE`

Three layers, stacked

Your server speaks HTTP. But HTTP rides on TCP, which rides on IP. Each layer trusts the one below it to handle the messy parts — so you only write the top one.

HTTP
HyperText Transfer Protocol

Text rules — **GET /todos** and a response. The layer we read, route and reply in. **We build this.**

TCP
Transmission Control Protocol

A reliable, ordered byte stream. Nothing lost, nothing scrambled. **The kernel handles this.**

IP
Internet Protocol

Raw packets across the network — may be lost, duplicated or arrive out of order.

TCP · TRANSMISSION CONTROL PROTOCOL

TCP is a phone call

You dial, they pick up, then you talk — words arrive in order, and you notice if the line drops. Four promises make a connection feel that reliable:

Reliable

Lost a packet? TCP notices and resends. Data arrives, or you get an error — never silent loss.

Ordered

Bytes come out in the exact order you sent them, even when packets took different routes.

A stream

A continuous pipe of bytes, not tidy messages. One read may not hand you a whole request.

Connected

A handshake first, so both sides agree they are talking. That is the listen / accept / connect dance.

TCP vs UDP

TCP

Transmission Control Protocol

A phone call

- Handshake before any data
- Resends anything lost
- Keeps bytes in order
- A continuous stream

Web, email, SSH — and our server.

UDP

User Datagram Protocol

Postcards in a mailbox

- No connection, no setup
- Lost packets stay lost
- May arrive out of order
- Discrete messages

Video calls, games, DNS.

In code it is one constant: **SOCK_STREAM** gives you TCP, **SOCK_DGRAM** gives you UDP. HTTP needs every guarantee in the blue column, so we pick TCP.

GETTING A SERVER LISTENING

Five system calls

1	`socket()`	Give me an endpoint — returns a file descriptor (just an int).
2	`bind()`	Attach it to an address and port — here, port 8080.
3	`listen()`	Start queuing incoming connections.
4	`accept()`	Block until a client connects, then hand me a new fd for that one client.
5	`read() / close()`	Read the bytes the client sent, then hang up.

Where TCP lives in the code

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
```

AF_INET

Use IPv4 addresses.

SOCK_STREAM

This line chooses TCP. Swap in **SOCK_DGRAM** and you get UDP instead.

So a “TCP socket” just means a socket created with **SOCK_STREAM** — one endpoint for a reliable, ordered, phone-call-style byte stream. Everything else we build is string handling on top.

htons · HOST TO NETWORK SHORT

Agreeing on byte order

Machines disagree on how to store a multi-byte number — this is **endianness**. The network picks one order (big-endian), so you convert before putting a port on the wire.

Port 8080 = 0x1F90

`big-endian → 1F 90`

network order: high byte first

`little-endian → 90 1F`

most laptops, incl. your Mac

The conversion family

htons host → net, short (port)

htonl host → net, long (IPv4)

ntohs net → host, short

ntohl net → host, long

use the **ntoh*** pair when reading values off the network

> Forget **htons** on a little-endian Mac and the bytes flip: you bind to **36895**, not 8080. Not decoration — correctness.

THE SERVER LOOP · PART 1

Everything is a file descriptor

A file descriptor (fd) is just an **int** — a ticket the OS hands you for something it opened. Files, sockets, pipes, the terminal: all the same kind of handle, all read and written the same way.

Always open before you start

fd	name	is
0	stdin	keyboard in
1	stdout	normal out
2	stderr	error out
3	server_fd	your socket

The accept() twist

server_fd is the receptionist — it only answers the door, and keeps listening.

Each **accept()** returns a **new** fd for one client conversation. You read and write *that* one, then close it.

THE SERVER LOOP · PART 2

while(1): one client at a time

The server never ends. It serves a client, then loops back for the next. Four beats, forever:

1	`accept()`	Sleep here until a client connects — this is what **blocking** means.
2	`read()`	Pull their request bytes into a buffer; null-terminate to make a string.
3	`write()`	Send the HTTP response back — status line, headers, blank line, body.
4	`close(client_fd)`	Hang up on *this* client — the listener stays open for the next.

↶ back to the top, forever

HTTP is just text

Both directions are plain text over the TCP stream: a first line, some **Key: Value** headers, a blank line, then an optional body. Handling it is string work — no magic.

REQUEST — received

```
GET /todos HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.7.1
— blank line ends the headers
```

RESPONSE — sent

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 26
{"message":"Hello TODOs!"}
```

2xx success · 200 OK

4xx your fault · 404 Not Found

5xx our fault · 500 Error

Where things live in memory

In C, you manage the whole lifecycle yourself. Three places data can live — and only one of them can leak.

Stack

Locals: **int opt**, **char buffer[4096]**.

Freed automatically on return.

Heap

Whatever you **malloc()**.

You free it — or it leaks.

Static

String literals, baked into the binary.

Lives forever; never freed.

const = read-only. It is about mutability, not leaks.

Every **malloc()** needs a matching **free()**.

C habits worth keeping

`call, then check < 0`	No exceptions in C — inspect every return value. −1 means failure.
`leave room for '\0'`	Read at most **size − 1**; a C string ends at its null byte.
`\r\n on the wire`	HTTP lines end with CRLF, whatever your OS uses for text files.
`measure → malloc → build`	**snprintf(NULL, 0, …)** gives the exact size for a dynamic string.
`-Wall -Wextra`	Let the compiler catch typos — it found your **&s** vs **%s** tonight.

READING A POST

Finding the body

The whole request is one buffer: request line, headers, a blank line, then the body. The blank line — **\r\n\r\n** — is the marker. Everything after it is the payload.

```
POST /todos HTTP/1.1
Content-Type: application/json
↩ blank line — \r\n\r\n
{"title":"Buy milk"}
```

```
char *body = strstr(buffer, "\r\n\r\n");
body += 4; // step past the separator
```

strstr returns a pointer *into* the same buffer — no copy. Adding 4 walks it past the four separator bytes to the first character of the JSON.

JSON is just a string

C has no concept of JSON. A todo lives as a **struct** in memory; JSON is the **text** it becomes at the edges. The real work is converting between the two.

STRUCT · in memory

```
typedef struct {
  int  id;
  char title[256];
  int  completed;
} Todo;
```

snprintf →

← parse

JSON · on the wire

```
{"id":1,
 "title":"Buy milk",
 "completed":false}
```

This is what **JSON.stringify** and **res.json()** hide in Node — here you build every brace, quote and comma yourself.

Parsing a value by walking pointers

To pull **Buy milk** out of {"title":"Buy milk"}, we hop from marker to marker with pointers, then copy what sits between the quotes.

`strstr(body,"\"title\"")`	find the key
`strchr(…, ':')`	then the colon
`strchr(…, '"') + 1`	opening quote → first char of value
`end - start`	pointer subtraction = the length, then memcpy

`''` = one char · `""` = a string. strchr wants a char.

Crude by design: no escapes, trusts well-formed input.

THE DATA MODEL

An in-memory store

The “database” is a fixed array of structs plus a count — declared at file scope so every handler can reach it.

```
Todo todos[MAX_TODOS]; // the store
int  todo_count = 0; // how many exist
int  next_id = 1; // next id to assign
```

Fixed array — chosen here

Size known at compile time → static memory, no malloc, no leaks. Capped at MAX_TODOS.

Why global?

Shared state every route reads and writes. A pragmatic simplification for one file.

Fixed size → static array. Unknown size → that is when malloc earns its place.

GET /todos — LISTING

Where malloc, 500 & free meet

The list's size depends on how many todos exist — a runtime unknown. So the three memory ideas from night one finally arrive together, in one handler.

malloc

Size unknown at compile time → allocate it at runtime.

500

If malloc returns NULL, the server failed → a real 500.

free

Release it after writing — or leak a little on every request.

Build the array by hand: a **pos** cursor walks the buffer — write **[**, then each todo with commas between, then **]**. No **.join()** in C.

The string toolkit

C has no string type with methods — just these functions over **char** arrays. **str** = string op; the suffix tells you the rest.

fn	does	returns
strstr	find a string in a string	pointer / NULL
strchr	find a char in a string	pointer / NULL
strcmp	compare two strings	0 if equal
strcpy	copy a string	—
strlen	length up to the '\0'	size_t
memcpy	copy N raw bytes	—
snprintf	build a string, size-capped	length it wanted

Every search returns a pointer or NULL — always check before you use it. snprintf is the one that builds: your stringify, interpolation and join in a single call.

THE FINISHED API

Six endpoints, full CRUD

The contract stays identical in every language to come. Only the implementation changes — this is the shape to re-target.

verb	path	does	ok
GET	/todos	list all	200
POST	/todos	create	201
GET	/todos/:id	fetch one	200
PATCH	/todos/:id	partial update	200
PUT	/todos/:id	full replace	200
DELETE	/todos/:id	remove	204

Errors are honest too: **400** malformed, **404** no such todo, **500** server failure. PATCH is lenient, PUT is strict — that is why both exist.

UUIDv4 FROM SCRATCH

16 random bytes, two forced

A UUID is 128 random bits shown as hex in an **8-4-4-4-12** pattern. Fill 16 bytes with real randomness, then stamp two positions to mark it version 4.

```
arc4random_buf(bytes, 16); // no rand(), no seeding
bytes[6] = (bytes[6] & 0x0F) | 0x40; // version 4
bytes[8] = (bytes[8] & 0x3F) | 0x80; // variant
```

clear & set: (v & mask) | bits — AND clears, OR sets.

%02x prints each byte as two hex digits, zero-padded.

7aeabf06-2f66-4742-9537-b239db7f1066

Routing & path parameters

Match the method and the path, then pull the id out of /todos/:id. Exact routes use a full compare; parameterized ones match a prefix.

Exact path

`strcmp(path, "/todos")`

the whole string must match

Prefix + param

`strncmp(path, "/todos/", 7)`

matches /todos/<anything>

```
char *id = path + 7; // skip "/todos/", point at the id
```

The prefix length and the **+7** must agree — off by one and the id gets a stray slash.

Then validate: is it a real UUID? → **400** if not.

Then find it in the store → **404** if missing.

CTRL-C, HANDLED

Graceful shutdown

Catch **SIGINT**, set a flag, let the loop finish and clean up — instead of dying mid-sentence. The handler does one safe thing: flip the flag.

`volatile sig_atomic_t`	a flag that can change asynchronously and is safe to set from a handler
`handler: running = 0`	do the minimum — most functions are not safe inside a signal
`while (running)`	loop exits on the flag — close(server_fd) is finally reachable

Use **sigaction**, not **signal**: the old call auto-restarts **accept()**, so Ctrl-C hung. No SA_RESTART → accept returns EINTR → the loop exits.

THE BIG LIMITATION

One request at a time

Our server is single-threaded and blocking: accept → handle → close, then the next. Fast here (microseconds each), but one slow request would stall the whole line.

Thread per connection

one thread each — simple, heavy at scale

Thread pool

fixed workers pull from a queue

Event loop

one thread, never blocks — the Node model

You feel the problem here, then get it for free next: **Go** runs each request in a goroutine, **Node** on an event loop — concurrency the runtime hands you.

SWAGGER, FROM C

Interactive docs, demystified

Auto-docs are not magic: a **static HTML page** (Swagger UI) that reads a **JSON spec**. Serve both from the server — one new route each.

`GET /openapi.json`

serve the spec, read from disk

`GET /docs`

serve the Swagger UI page

First file I/O: fopen → fseek/ftell (size) → malloc → fread → fclose

Same acquire-use-release rhythm as sockets — a new resource, patterns you already know.

C can't **introspect** itself, so the spec is hand-written. FastAPI reads your type hints; here you write the JSON — now you know what the framework automates.

DONE · IN C

A complete REST API

- ✓ TCP server with a full lifecycle
- ✓ HTTP parsed & built by hand
- ✓ Six CRUD endpoints, correct verbs & codes
- ✓ JSON & a UUIDv4 generator from scratch
- ✓ Manual memory, validation, graceful shutdown

SAME CONTRACT, NEXT LANGUAGE

Re-target the identical API and watch the code shrink as the runtime does more.

C++ · Rust · Go
Node · Express · Nest
PHP · Laravel

The hard version is the valuable one: you built what every framework hides, so the next ones will feel like gifts — and you will know exactly what they do for you.